

שרת Express לחנות אונליין

תוכן עניינים

1. מטרת הפרויקט

- בניית routes עם Express.
- קריאת נתונים מתוך query params.
- קריאת נתונים מתוך request body (JSON).
- החזרת JSON responses עם status codes מתאימים.
- ניהול קונפיגורציה של הפרויקט עם קובץ .env.
- שמירה וקריאה של נתונים מקבצי JSON.

2. תיאור הפרויקט

שרת Backend שמדמה חנות אונליין קטנה. אתם בוחרים איזה סוג חנות תרצו לבנות, לדוגמה:

- חנות בגדים
- חנות נעליים
- חנות צעצועים
- חנות ספרים
- חנות אופטיקה (משקפיים)

החנות מנהלת מוצרים, עגלת קניות, ותהליך checkout — אבל בשלב הזה, בלי מערכת התחברות. כדי לזהות "לקוח", כל בקשה שקשורה לעגלה או ליתרה תכלול customerId מפורש (כפרמטר או כשדה ב-body) — ראו הרחבה בפרק 5.

3. Environment Variables — קובץ .env

מידע קונפיגורציה, ובפרט הנתיב לתיקיית ה-database, לא נכתב ישירות בתוך הקוד. הוא נשמר בקובץ .env מקומי, שלא נכנס ל-Git.

דוגמה לתוכן קובץ .env:

```
PORT=3000
DB_BASE_PATH=./db
STARTING_BALANCE=500
```

- PORT — הפורט שעליו השרת מאזין.
- DB_BASE_PATH — הנתיב לתיקייה שבה שמורים קבצי ה-JSON של הנתונים.
- STARTING_BALANCE — יתרת הכסף ההתחלתית שניתנת ללקוח חדש.
- קובץ .env עצמו חייב להופיע ב-gitignore ולא להיכנס ל-repository.
- כל מקום בקוד שצריך את הנתיב לתיקיית ה-database, את הפורט או את יתרת הפתיחה — קורא אותם מתוך process.env, ולא כותב אותם ישירות (hardcoded).

4. מודל הנתונים

מוצר (Product))

הסבר	שדה
מזהה ייחודי של המוצר.	id
שם המוצר.	name
מחיר המוצר.	price
כמות במלאי.	stock

המוצרים מוגדרים מראש על ידיכם, בקובץ JSON. משתמשים לא יוצרים מוצרים חדשים בשלב הזה.

לקוח (Customer))

הסבר	שדה
מזהה לקוח — נשלח על ידי הקליינט בכל בקשה רלוונטית (ראו הערה למטה).	customerId
יתרת כסף נוכחית של הלקוח.	balance
מערך פריטים בעגלה — כל פריט כולל productId ו-quantity.	cart
תאריך יצירת רשומת הלקוח.	createdAt

הזמנה (Order))

הסבר	שדה
מזהה ההזמנה.	id
מזהה הלקוח שביצע את ההזמנה.	customerId
רשימת הפריטים שנקנו, עם כמות ומחיר.	items
סכום כולל של ההזמנה.	total
תאריך יצירת ההזמנה.	createdAt

5. נתיבי API נדרשים

Method	נתיב	תיאור
GET	/	הודעת פתיחה קצרה על ה-API.
GET	/health	בדיקת תקינות לשרת.
GET	/products	רשימת מוצרים, עם סינון (ראו פרק 7).
GET	/cart	מציג את עגלת הקניות של customerId נתון.
POST	/cart/items	מוסיף מוצר לעגלה של הלקוח.
DELETE	/cart/items/:productId	מסיר מוצר מהעגלה של הלקוח.
GET	/account/balance	מציג את היתרה הנוכחית של הלקוח.
POST	/orders/checkout	מבצע checkout ויוצר הזמנה חדשה.
GET	/orders	מציג את היסטוריית ההזמנות של הלקוח.

כל נתיב שלא מוגדר ברשימה זו צריך להחזיר 404.

6. Query Params נדרשים

סינון מוצרים — GET /products

דוגמה	הסבר	פרמטר
?inStock=true	מחזיר רק מוצרים שיש מהם מלאי (stock גדול מ-0).	inStock
?maxPrice=50	מחזיר מוצרים שהמחיר שלהם קטן או שווה לערך.	maxPrice
?search=gllasses	חיפוש טקסט חופשי בתוך שם המוצר.	search

הפרמטרים צריכים לעבוד גם בשילוב זה עם זה, לדוגמה:

```
GET /products?inStock=true&maxPrice=50&search=gllasses
```

זיהוי לקוח — GET /cart, GET /account/balance, GET /orders

בקשות ה-GET האלה חייבות לקבל את customerId כ-query param. לדוגמה:

```
GET /cart?customerId=abc123GET /account/balance?customerId=abc123GET /orders?customerId=abc123
```

- אם customerId חסר בבקשה — יש להחזיר 400.

7. Body בבקשות

יש לקרוא JSON body בעזרת `express.json()` בנתיבים הבאים:

POST /cart/items

```
{ "customerId": "abc123", "productId": 1, "quantity": 2}
```

- ו-customerId, productId ו-quantity הם שדות חובה.
- quantity חייב להיות מספר שלם גדול מ-0.
- אם המוצר לא קיים — 404.
- אם אין מספיק מלאי — 400.

DELETE /cart/items/:productId

```
{ "customerId": "abc123"}
```

- productId מגיע מה-route param (URL), ו-customerId מגיע מה-body.

POST /orders/checkout

```
{ "customerId": "abc123"}
```

- דוחה checkout על עגלה ריקה — 400.
- בודק שכל המוצרים בעגלה עדיין קיימים ושיש מספיק מלאי — אחרת 400.
- בודק שליתרת הלקוח מספיק כסף לתשלום — אחרת 400.
- אם ה-JSON שנשלח אינו תקין — יש להחזיר 400 Bad Request.

8. כללי עסק חשובים

- מוצרים גלויים לכולם — GET /products לא דורש customerId.
- אפשר להוסיף לעגלה רק מוצר שקיים במלאי מספיק.
- המלאי בפועל יורד רק בזמן checkout מוצלח — לא בזמן ההוספה לעגלה.

- checkout מצליח רק אם יש ללקוח מספיק כסף ביתרה.
- היתרה של לקוח משתנה רק כתוצאה מ-checkout מוצלח — אין נתיב שמאפשר להוסיף כסף באופן ידני.
- לקוח חדש (customerId שלא נראה בעבר) מקבל אוטומטית את יתרת הפתיחה מה-env.

9. מבנה תשובות ו-Status Codes

הצלחה:

```
{ "success": true, "data": {} }
```

שגיאה:

```
{ "success": false, "message": "Error message" }
```

מתי משתמשים בו	קוד
בקשה שהצליחה (GET, DELETE, checkout וכו').	200
בקשה לא תקינה, ולידציה נכשלה, או JSON שגוי.	400
נתיב, מוצר או הזמנה שלא נמצאו.	404
שגיאה פנימית לא צפויה בשרת.	500

10. תרחישי בדיקה מומלצים

- GET /health — מחזיר תקין.
- GET /products — מחזיר את כל המוצרים.
- GET /products?inStock=true&maxPrice=50 — בדיקת סינון משולב.
- POST /cart/items עם body תקין — מוצר נוסף לעגלה.
- POST /cart/items על מוצר שאין ממנו מספיק מלאי — מצפים ל-400.
- GET /cart?customerId=... — מחזיר את העגלה עם totals.
- GET /cart בלי customerId — מצפים ל-400.
- DELETE /cart/items/:productId — מסיר פריט מהעגלה.
- POST /orders/checkout על עגלה ריקה — מצפים ל-400.
- POST /orders/checkout כשאין מספיק כסף ביתרה — מצפים ל-400.
- POST /orders/checkout מוצלח — בדיקה שהיתרה, המלאי וההזמנות התעדכנו.
- GET /account/balance?customerId=... — מחזיר את היתרה הנוכחית.
- GET /orders?customerId=... — מחזיר רק את ההזמנות של אותו לקוח.
- בקשה לנתיב שלא קיים — מצפים ל-404.

11. הגשת הפרויקט

- יש להגיש קישור ל-repository ב-GitHub, עם היסטוריית commits ברורה.
- קובץ env. לא נכנס ל-repository — רק env.example.
- יש לצרף README קצר שמסביר איך מריצים את הפרויקט ואילו נתיבים קיימים.